BUSINESS FINAL REPORT

Casestudy -
supply_chain_data

Submitted by: Goli Bhagyasri

TABLE OF CONTENTS

List of Figures
Fig1. First five rows of data
Fig2. Statistical analysis of data
Fig3. Product wg by location type
Fig4. Product wg by zone
Fig5. Product wg by temp reg match
Fig6. Product wg by approved_wh_govt_Certificate
Fig7. Product wg by warehouse age
Fig8. Product wg by storage_issue_reported_l3m
Fig9. Product wg by wh_breakdown_l3m
Fig10. Heatmap of multiple variables
Fig11. Visualizing the feature importances
Fig12. Observed vs predicted – XGB Tuned model

Supply Chain Data - FMCG in Instant noodles

Business Context:

A fast-moving consumer goods (FMCG) company that entered the instant noodles market two years ago is facing challenges in balancing demand and supply across its warehouses nationwide. In some regions, demand far exceeds supply, while in others, excess inventory is leading to wastage and higher costs. This mismatch results in significant inventory losses and missed sales opportunities. To remain competitive and profitable in the highly dynamic FMCG sector, the company needs a data-driven approach to optimize its supply chain and align production with regional demand patterns.

Objective:

As a data scientist, you are tasked with building a predictive model using historical sales and supply data to determine the optimum quantity of product to ship to each warehouse. Additionally, analyzing demand patterns across different regions will help management design targeted advertisement campaigns. Solving this problem will enable the company to minimize inventory costs, reduce stockouts, and drive higher sales through better demand forecasting and efficient supply chain management

Data Dictionary:

- The dataset contains information about the historical sales patterns. The detailed data dictionary is given below:
- Ware_house_ID: Product warehouse ID
- WH_Manager_ID: Employee ID of warehouse manager
- Location_type: Location of the warehouse, like in a city or a village
- WH_capacity_siz: Storage capacity size of the warehouse
- zone: Zone of the warehouse
- WH_regional_zone: Regional zone of the warehouse under each zone
- num_refill_req_l3m: Number of times refilling has been done in the last 3 months
- transport_issue_l1y: Any transport issue, like an accident or goods stolen, reported in the last year
- Competitor_in_mkt: Number of instant noodles competitors in the market
- retail_shop_num: Number of retail shops that sell the product under the warehouse area
- wh_owner_type: Company owns the warehouse, or they have rented the warehouse
- distributor_num: Number of distributors working between the warehouse and retail shops
- flood_impacted: Warehouse is in the flood-impacted area indicator
- flood_proof: Warehouse is flood-proof. Like storage is at some height, not directly on the ground
- electric_supply: The Warehouse has an electric backup, like a generator, so they can run the warehouse in load shedding
- dist_from_hub: Distance between the warehouse to the production hub in Kms
- workers_num: Number of workers working in the warehouse
- wh_est_year: Warehouse established year
- storage_issue_reported_l3m: Warehouse reported a storage issue to the corporate office in the last 3 months. Like rats, fungus due to moisture, etc.
- temp_reg_mach: Warehouse has a temperature regulating machine indicator

- approved_wh_govt_certificate: What kind of standard certificate has been issued to the warehouse from the government regulatory body
- wh_breakdown_l3m: Number of times the warehouse faced a breakdown in the last 3 months. Like a strike by workers, a flood, or an electrical failure
- govt_check_l3m: Number of times government officers have visited the warehouse to check the quality and expiry of stored food in the last 3 months
- product_wg_ton: Product has been shipped in the last 3 months. Weight is in tons

Key Point to Analyse:

Analyze the data and come up with a linear regression model to determine the driving factors for product volumes

MODULES DEVELOPED:

Python libraries used
Numpy, Pandas, Matplotlib, Seaborn, Scipy.stats, sklearn.model_selection, sklearn.metrics, statsmodels.api, statsmodels.stats.outliers_influence, sklearn.linear_model

DATA OVERVIEW:

- The dataset contains information about supply chain data of a FMCG organisation.
- The data contains Data has 25000 rows and 24 columns.
- The data seems to show missing values in 3 attributes namely workers_num, wh_est_year, approved_wh_govt_certificate.
- Numerical variables include total 14 variables, num_refill_req_l3m,transport_issue_l1y, Competitor_in_mkt,retail_shop_num, distributor_num, flood_impacted , flood_proof, electric_supply, dist_from_hub, storage_issue_reported_l3m, temp_reg_mach, wh_breakdown_l3m, govt_check_l3m, product_wg_ton
- Categorical columns include total 8 variables namely ware_house_ID, WH_Manager_ID, Location_type, WH_capacity_size, zone, WH_regional_zone, wh_owner_type, approved_wh_govt_certificate
- Float variables are workers_num,wh_est_year which can be changed to Date time format

	Ware_house_ID	WH_Manager_ID	Location_type	WH_capacity_size	zone	WH_regional_zone	num_refill_req_l3m	transport_issue_l1y	Competitor_in_mkt	reta
0	WH_100000	EID_50000	Urban	Small	West	Zone 6	3	1	2	
1	WH_100001	EID_50001	Rural	Large	North	Zone 5	0	0	4	
2	WH_100002	EID_50002	Rural	Mid	South	Zone 2	1	0	4	
3	WH_100003	EID_50003	Rural	Mid	North	Zone 3	7	4	2	
4	WH_100004	EID_50004	Rural	Large	North	Zone 5	3	1	2	

Fig1. First five rows of data

STATISTICAL SUMMARY:

- high variance in retail_shop_num, product_wg_ton, and storage_issue_reported_l3mBinary features like electric_supply and flood_impacted may act for segmentation flags
- Right-skewed target may benefit from log transformation for regression modeling

	count	mean	std	min	25%	50%	75%	max
num_refill_req_l3m	25000.0	4.089040	2.606612	0.0	2.0	4.0	6.0	8.0
transport_issue_l1y	25000.0	0.773680	1.199449	0.0	0.0	0.0	1.0	5.0
Competitor_in_mkt	25000.0	3.104200	1.141663	0.0	2.0	3.0	4.0	12.0
retail_shop_num	25000.0	4985.711560	1052.825252	1821.0	4313.0	4859.0	5500.0	11008.0
distributor_num	25000.0	42.418120	16.064329	15.0	29.0	42.0	56.0	70.0
flood_impacted	25000.0	0.098160	0.297537	0.0	0.0	0.0	0.0	1.0
flood_proof	25000.0	0.054640	0.227281	0.0	0.0	0.0	0.0	1.0
electric_supply	25000.0	0.656880	0.474761	0.0	0.0	1.0	1.0	1.0
dist_from_hub	25000.0	163.537320	62.718609	55.0	109.0	164.0	218.0	271.0
workers_num	24010.0	28.944398	7.872534	10.0	24.0	28.0	33.0	98.0
wh_est_year	13119.0	2009.383185	7.528230	1996.0	2003.0	2009.0	2016.0	2023.0
storage_issue_reported_l3m	25000.0	17.130440	9.161108	0.0	10.0	18.0	24.0	39.0
temp_reg_mach	25000.0	0.303280	0.459684	0.0	0.0	0.0	1.0	1.0

Fig2. Statistical analysis of data

Checking the data types:

- The data seems to show **missing values** in **3** attributes namely workers_num, wh_est_year, approved_wh_govt_certificate.
- **Numerical variables** include total **14** variables, num_refill_req_l3m,transport_issue_l1y, Competitor_in_mkt,retail_shop_num, distributor_num, flood_impacted , flood_proof, electric_supply, dist_from_hub, storage_issue_reported_l3m, temp_reg_mach, wh_breakdown_l3m, govt_check_l3m, product_wg_ton
- **Categorical columns** include total **8** variables namely ware_house_ID, WH_Manager_ID, Location_type, WH_capacity_size, zone, WH_regional_zone, wh_owner_type, approved_wh_govt_certificate
- Float variables are workers_num,wh_est_year which can be changed to Date time format

Checking for duplicates in the data:

- There seems to be no duplicate values in the data

Checking and Treating Missing values in the Data

- There seems to be 3 features showing the missing values in the data, workers_num,wh_est_year, approved_wh_govt_certificate

- These can be replaced with mode for categorical variables like approved_wh_govt_Certificate and median/central value for numerical variables like workers_num and wh_est_year

Treating Missing values in the Data

- Now the features are imputed bases on KNN imputer.
- Likewise we convert year to age of warehouse
- We also test the significance of warehouse age against the product weight. Its shows 54% correlation which means positive, as the warehouse age increases, the more volumes are dispatched.
- However we check for remaining missing values and see approved_wh_gov_certificate has some which need to be adjusted.

Changing the Data types

- Now we change the data types of workers number to numerical for better analysis.
- Also we check if all variables show correct datatypes and count to proceed for data modelling.

NOTE:

Now the data seems clean, missing values are imputed with median and modes respectively. No duplicates are seen. And the data types are also changed as per the variable nature.

Data Standardization/Preprocessing

Let's check the count of each unique category in each of the categorical variables.

Ware_house_ID: 25000 unique values
WH_Manager_ID: 25000 unique values
Location_type: 2 unique values
WH_capacity_size: 3 unique values
zone: 4 unique values
WH_regional_zone: 6 unique values
wh_owner_type: 2 unique values
approved_wh_govt_certificate: 5 unique values

The Ware_house_ID and WH_Manager_ID columns will not add any value to our analysis so let's drop it before we move forward.

EXPLORATORY DATA ANALYSIS

Univariate and Bivariate analysis of Categorical variables:

Categorical columns : Location_type', 'WH_capacity_size', 'zone', 'WH_regional_zone', 'wh_owner_type', 'flood_impacted', 'flood_proof', 'electric_supply', 'temp_reg_mach', 'approved_wh_govt_certificate'

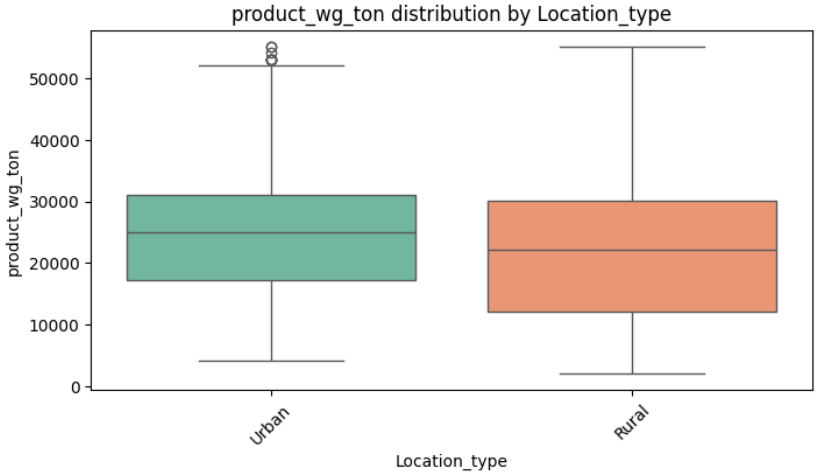


Fig3. Product wg by location type

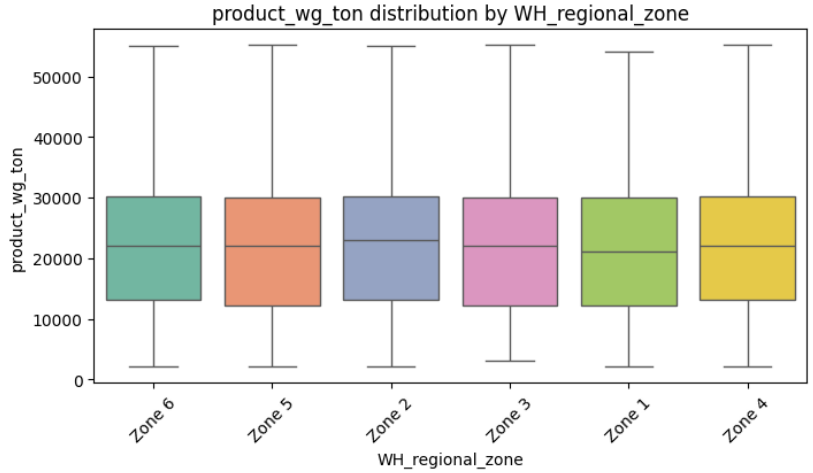


Fig4. Product wg by zone

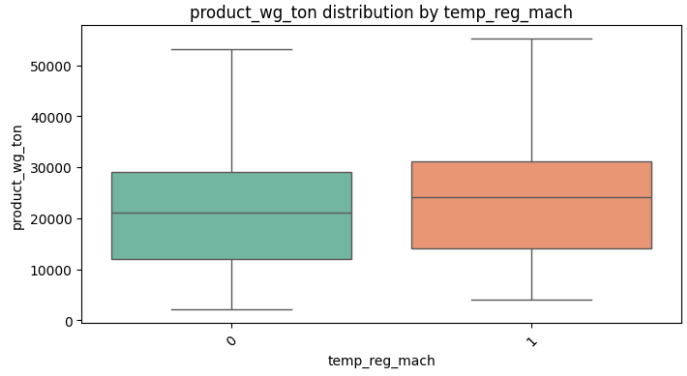


Fig5. Product wg by temp reg match

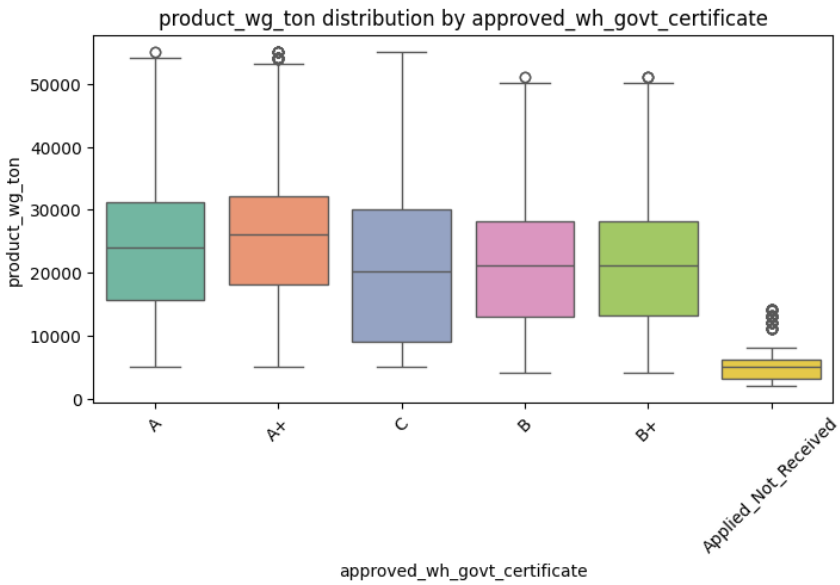


Fig6. Product wg by approved_wh_govt_Certificate

- Warehouse ownership type, Wh capacity size, distribution by Zone, Wh owner type, flood impacted, flood proof, distribution by electric supply are the variables which does not strongly influence product weight — since the categories have similar medians and distributions.
- Lets us also check for their significance with target variable by ANOVA test. - ANOVA tests whether the mean product weight differs significantly across categories (e.g., flood-proof vs not flood-proof warehouses)
- **Location type, Regional Zone, temp reg mach, approved wh govt certificate** – show strong separation when compared with product weight. The medians and means show a high variability and skewdness.

ANOVA Test of Significance between categorical variables and Target variable:

- These categorical variables do not impact product weight in the dataset.
- Including them in the predictive model would add noise rather than useful signal and hence it is safe to drop them from modeling.

Univariate and Bivariate analysis for Numeric Variables

Fig7. Product wg by warehouse age

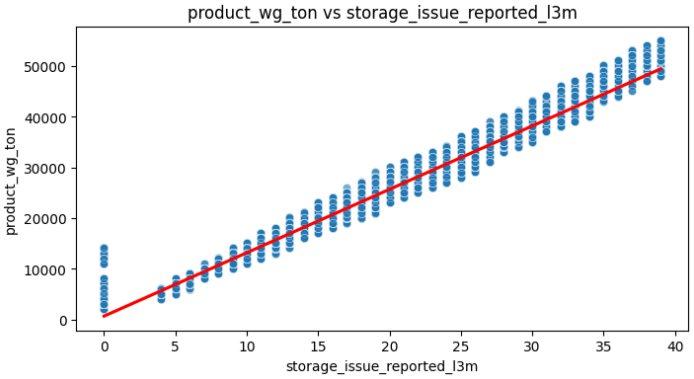


Fig8. Product wg by storage_issue_reported_l3m

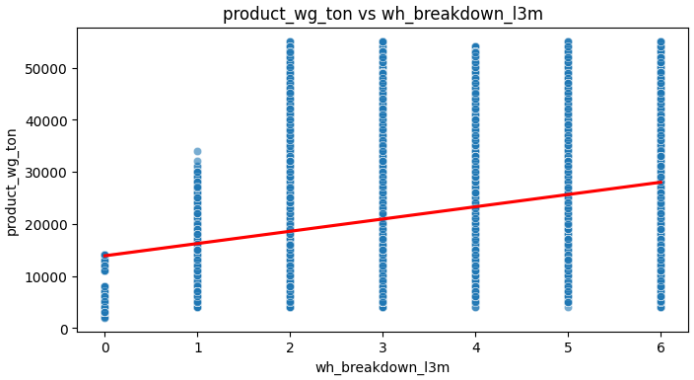


Fig9. Product wg by wh_breakdown_l3m

- Num refill re l3m, competitor in market, retail shop num, distributor num, dist from hub, workers num, govt check l3m – these variables show no relationship with product weight as the graphs does not show any linear trend.
- Transport issue l1y – downtrend or an inverse relationship – warehouses with more transport issues tend to handle lighter products. This may be due to an anticipation that heavy products need better transport.
- Warehouse age, storage issue reported l3m, wh breakdown l3m – these variables show significant linear trend with product weight and are the useful predictors for product weight.

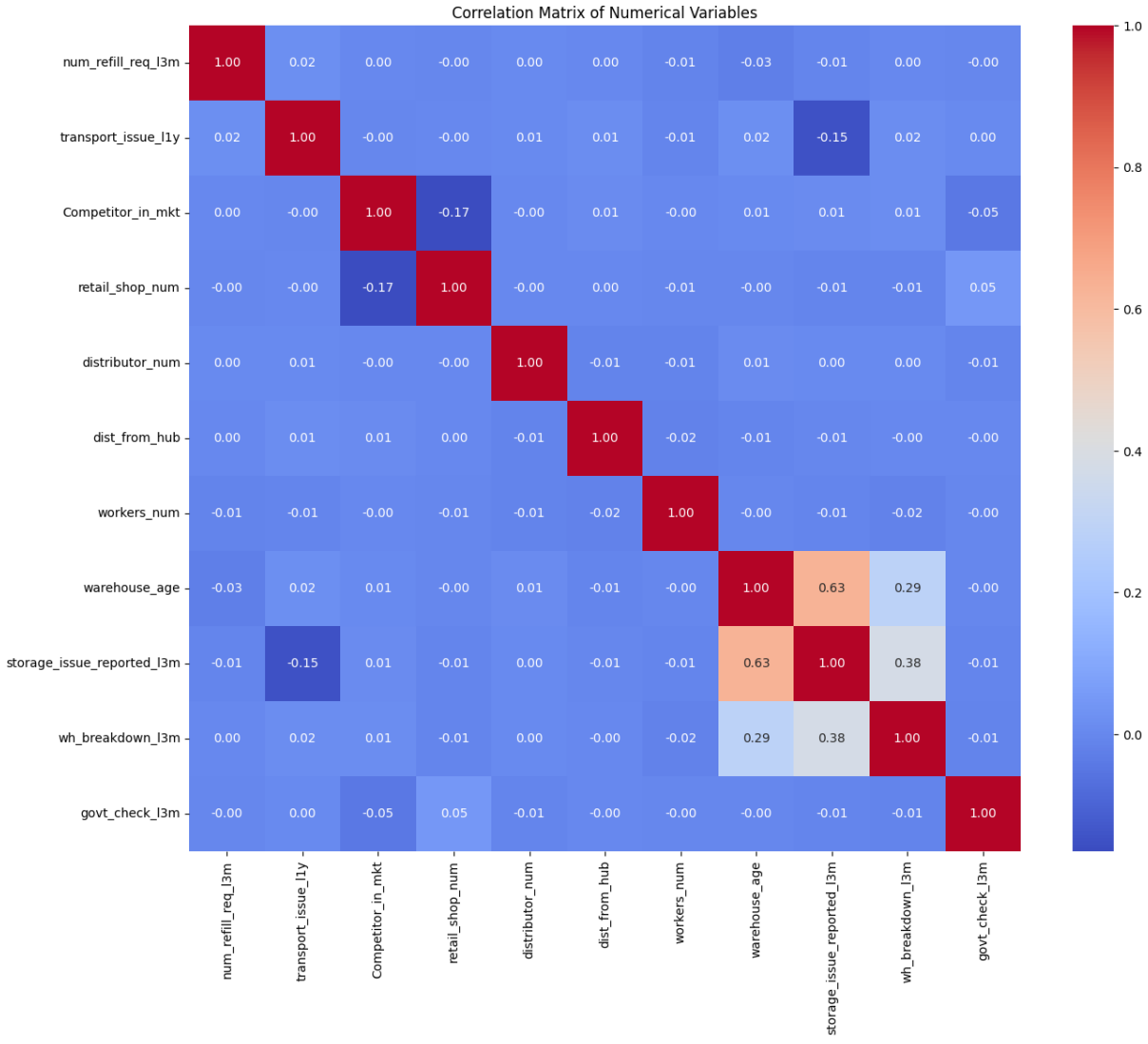
- Strongest predictors: storage_issue_reported_l3m and warehouse_age show the most pronounced positive correlation with product_wg_ton.
- Operational stress indicators: Features like wh_breakdown_l3m and num_refill_req_l3m may indirectly signal higher product volumes.
- Weak or neutral predictors: transport_issue_l1y and Competitor_in_mkt show little to no relationship with the target.

Outlier treatment:

- Outliers are seen in Location_type and approved_wh_govt_certificate variables.
- They can be treated through IQR method or log transformation method.
- However these are the real data that cannot be treated as they significantly impact

Multivariate Analysis

Fig10. Heatmap of multiple variables



Multivariate analysis helps us understand the relationships between multiple variable Simultaneously. We can analyse that with a heat map and a pair plot.

- Positive correlation : product_wg_ton ↔ storage_issue_reported_l3m = 0.99. This

means warehouses which ship more products tend to report more storage issues and vice versa.

- Negative correlation : wh_est_year ↔ storage_issue_reported_l3m = −0.63. This indicates that the newer the warehouse, less the storage issues and the older the warehouse the more the storage issues.
- Negative correlation : wh_est_year ↔ product_wg_ton = −0.60. This indicates that the newer the warehouse, lower quantity of product is shipped and the older the warehouse the more quantity of product is shipped.

Note: This indicates that older warehouses ship more product and report more storage issues whereas newer warehouses ship less product and report less storage issues

Encoding Categorical Variables by Onehot encoding

- Each category in a categorical feature is transformed into a new binary column.
- The column takes the value 1 if the observation belongs to that category, otherwise 0.
- This avoids assigning arbitrary numeric values (like 1, 2, 3) that could mislead models into thinking categories have an order.

Model Building:

Target Variable:

product_wg_ton - Quantity shipped in last 3 months

Modelling Goal:

Predict optimal shipment quantity per warehouse given:

- Infrastructure
- Market competition
- Logistics risks
- Regional characteristics

This is a supervised regression problem.

Defining X and Y(Target)

X = categorical_cols = one-hot columns

y = product weight

Train–Test Split

splitting the data in 70:30 ratio for train to test data

Feature scaling

Using standard scaler we transform the data and we observe:

Number of rows in train data = 17500

Number of rows in test data = 7500

We choose 5 models for model building to check the performance of data.

1. Linear regression model
2. Decision tree model
3. Random forest model
4. Bagging model
5. XG Boost model

1.Baseline Model – Linear Regression

Linear Regression was chosen as a baseline due to its simplicity, interpretability, and effectiveness as a reference point for more complex models

We are predicting the demand through a simple linear regression model initially

- Model explains 99% variance when compared with test data to original data,this is exceptionally high.
- High R2 mean there might be an information breach.

Calculating both Train and Test MAE and RMSE:

- Train MAE = 883 tons / Test MAE = 875 tons - Very close, showing strong generalization.
- Train RMSE = 1150 tons / Test RMSE = 1113 tons - Slightly higher than MAE, as expected.
- Train MAPE = 6.16% / Test MAPE = 6.06% - Predictions are, on average, within ~6% of actual product weights. This is excellent accuracy for forecasting.
- R^2 (0.9903): The model explains ~99% of the variance in shipment quantities.
- Adjusted R^2 (0.99024): Almost identical to R^2, meaning the predictors are genuinely useful and not just increasing the score

Note: MAPE was chosen as the primary metric because percentage error is more interpretable.

OLS achieved R square = 0.99, MAE = 875 tons, RMSE = 1113 tons, MAPE = 6% and R2 = 99%, Train and test errors are consistent, showing strong generalization.

2. Building Decision Tree model

Results:
Decision Tree Train MAE: 74.00909430373919
Decision Tree Train RMSE: 407.74283438860886
Decision Tree Train MAPE (%): 0.8473174393108147
Decision Tree Train R^2: 0.9987739577746385
Decision Tree Train Adjusted R^2: 0.9987728357317622
Decision Tree Test MAE: 792.5922235357496
Decision Tree Test RMSE: 1158.43281732236
Decision Tree Test MAPE (%): 4.734361283715063
Decision Tree Test R^2: 0.9898875596111664
Decision Tree Test Adjusted R^2: 0.9898659373946461

Insights:

- MAE(74.0): On average, predictions are off by only ~74 tons in training — extremely low error.
- RMSE (407.7): Errors are small, confirming the model fits training data very tightly.
- MAPE (0.85%): Less than 1% deviation from actual values — near-perfect fit.
- R^2 (0.9988) & Adjusted R^2 (0.99877): The model explains almost all variance in training data.

This is a sign of overfitting: the model memorized training data almost perfectly

3. Building Random forest model

Results:
Random Forest Train MAE: 297.38351819227336
Random Forest Train RMSE: 513.0437368009373
Random Forest Train MAPE (%): 2.0600477820817567
Random Forest Train R^2: 0.9980589285007697
Random Forest Train Adjusted R^2: 0.9980571520811627
Random Forest Test MAE: 677.1176143262364
Random Forest Test RMSE: 918.4714715282316
Random Forest Test MAPE (%): 4.153889133331057
Random Forest Test R^2: 0.9936430965871589
Random Forest Test Adjusted R^2: 0.9936295043842182

Insights:

- Train MAE ≈ 297 tons / Train MAPE ≈ 2.06% → Very low training error, showing the forest fits training data strongly.
- Test MAE ≈ 677 tons / Test MAPE ≈ 4.15% → Test error is higher than train but still better than OLS (MAPE ≈ 6%) and Decision Tree (MAPE ≈ 4.73%).
- RMSE gap (Train ≈ 513 vs Test ≈ 918)
- R2 for both test and train data explains nearly 99% of variance. → Some overfitting, but much less severe than the single Decision Tree
- "Random Forest improved generalization compared to a single Decision Tree. Test MAPE dropped to 4.15%, outperforming the OLS baseline (~6%). The train–test gap is moderate, suggesting the ensemble stabilizes predictions while reducing overfitting.

4. Building Bagging model

Results:
Bagging Train MAE: 297.5265703794719
Bagging Train RMSE: 513.078910421441
Bagging Train MAPE (%): 2.060314938183841
Bagging Train R^2: 0.9980586623369018
Bagging Train Adjusted R^2: 0.9980568856737084
Bagging Test MAE: 677.0502954732042
Bagging Test RMSE: 918.3387195035842
Bagging Test MAPE (%): 4.155758348717517
Bagging Test R^2: 0.9936449340548178
Bagging Test Adjusted R^2: 0.9936313457807134

Insights:

- MAE: Training error is very low (~ 298 tons), while test error rises (~ 677 tons), showing some generalization gap but still acceptable.
- RMSE: Training deviations are small (~ 513), test deviations larger (~918), indicating higher variance on unseen data.
- MAPE: Training predictions deviate by ~2%, test predictions by ~4%, reflecting strong but slightly reduced accuracy on new data.

- R^2: Training fit is near perfect (0.998), test fit remains excellent (0.994), confirming the model explains almost all variance.
- Adjusted R^2: Training adjusted R^2 (0.99806) and test adjusted R^2 (0.99363) closely mirror R^2, validating that predictors are genuinely useful without overinflation

Bagging strikes a balance: it captures nonlinear relationships better than Linear Regression, avoids the extreme overfitting of a single Decision Tree, and performs comparably to Random Forest.

5. Building XG Boost model

Results:
XGBoost Train MAE: 603.8773193359375
XGBoost Train RMSE: 820.3350687371594
XGBoost Train MAPE (%): 3.764584749922916
XGBoost Train R^2: 0.995037317276001
XGBoost Train Adjusted R^2: 0.9950327755541235
XGBoost Test MAE: 649.2354125976562
XGBoost Test RMSE: 857.7701833824722
XGBoost Test MAPE (%): 4.0172791587631105
XGBoost Test R^2: 0.9944555759429932
XGBoost Test Adjusted R^2: 0.9944437209670595

Insights:
- MAE: Training error is ~604 tons, test error ~649 tons, showing consistently low average prediction error.
- RMSE: Training deviations are ~820, test deviations ~858, indicating stable variance across seen and unseen data.
- MAPE: Training predictions deviate by ~3.8%, test predictions by ~4.0%, reflecting strong accuracy with minimal drift.
- R^2: Training fit is 0.995, test fit 0.994, confirming the model explains nearly all variance in shipment quantities.
- Adjusted R^2: Training adjusted R^2 (0.99503) and test adjusted R^2 (0.99444) closely mirror R^2, validating predictor usefulness without inflation
- Very small gap, showing excellent balance between bias and variance. “XGBoost achieved the lowest test error among all models so far, with MAPE ≈ 4%. This indicates strong predictive accuracy and stability, outperforming both OLS and Random Forest. The small train–test gap suggests the model generalizes well without severe overfitting.

Comparing all the models to check the best performance:

- Overall when compared with all models XGBoost consistently ranks best across MAE, RMSE, MAPE, R^2, and Adjusted R^2 on test data.
- It balances flexibility and generalization better than other models.
- However we would like to hypertune three best models and observe the results.
- The best three models are XGBOOST, Random forest, Bagging.

Hyperparameter tuning:

Typical Hyperparameters to Tune in the models - **XGBOOST, Random forest, Bagging.**

- max_depth (limit tree depth)
- min_samples_split, min_samples_leaf (control branching)
- n_estimators (number of trees)
- One can use GridSearchCV or RandomizedSearchCV from scikit-learn

Best Params Selected:
XGBoost = 'colsample_bytree': 1.0, 'learning_rate': 0.0.
Random Forest = 'max_depth': None, 'max_features': 'sqrt'
Bagging = 'bootstrap': True, 'max_features': 1.0

Best Score (MAE)
XGBoost - 659.907104
Random Forest - 913.47348
Bagging - 693.489914

Insights:

- XGBoost remains the best performer after tuning, with the lowest MAE (~660).
- Bagging comes second (~693), very close to XGBoost.
- Random Forest lags behind (~913), showing higher error even after tuning.

Best Model: XGBoost It consistently delivers the lowest error and strongest generalization, making it the prime candidate for deployment or further fine-tuning

- **Retraining tuned XG boost on full training set:**

Results:
Tuned XGBoost Train MAE: 614.2919921875
Tuned XGBoost Train RMSE: 832.197354898464
Tuned XGBoost Train MAPE (%): 3.822162940821558
Tuned XGBoost Train R^2: 0.9948927760124207
Tuned XGBoost Train Adjusted R^2: 0.9948881020100296
Tuned XGBoost Test MAE: 634.1755981445312
Tuned XGBoost Test RMSE: 839.2019274286731
Tuned XGBoost Test MAPE (%): 3.881781108733913
Tuned XGBoost Test R^2: 0.9946930408477783
Tuned XGBoost Test Adjusted R^2: 0.9946816936145249

Insights:

- The tuned XGBoost model is highly accurate and generalizes extremely well.
- Train vs Test metrics are very close → no overfitting.
- With MAE ~634 and MAPE ~3.9% on test data, this is a strong model for deployment.

Feature importance

- Our model is highly dependent on one feature (storage_issue_reported_l3m).
- This can be a sign of strong predictive power of that feature and potential risk of over-reliance.

Visualizing the feature importances:

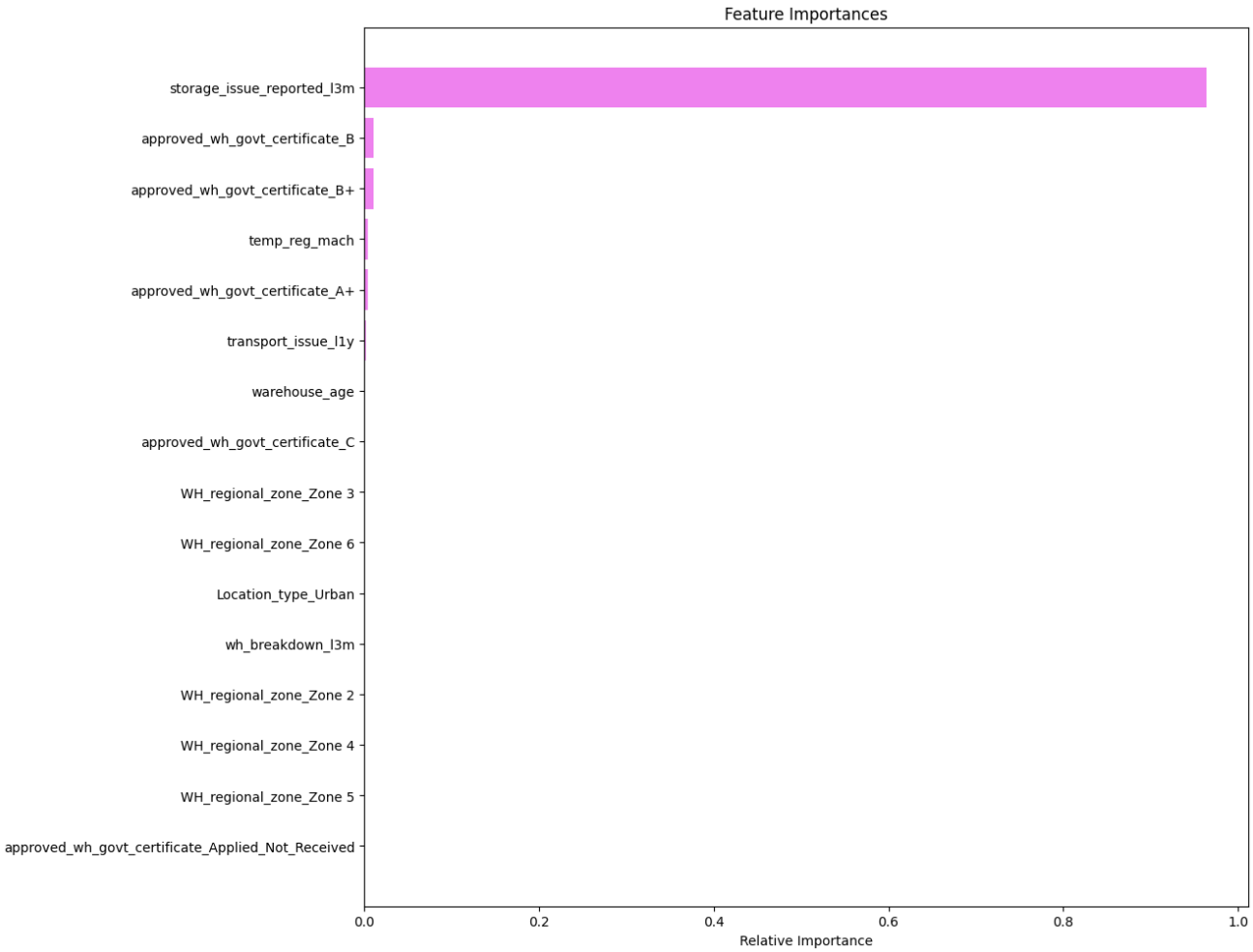


Fig11. Visualizing the feature importances

Insights:

- Our model is highly reliant on one feature, which can be both a strength and a risk:
- If that feature is stable and well-measured, it drives excellent performance.
- If it's noisy, missing, or changes over time, the model may degrade quickly

Observed VS Predicted Values Plot of the test data for the best model, i.e., XGBOOST tuned Model

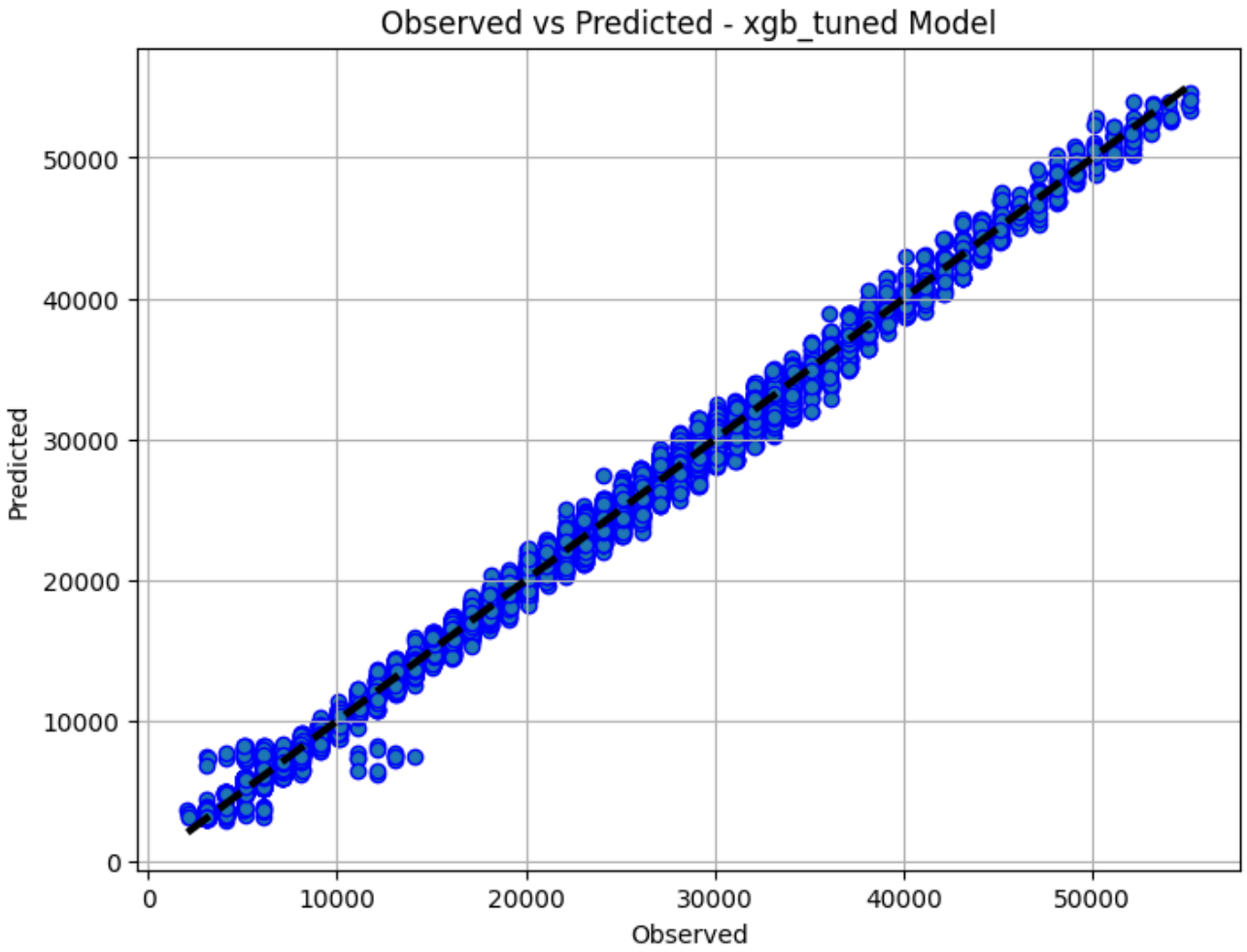


Fig12. Observed vs predicted – XGB Tuned model

The tight clustering of points around the diagonal line indicates:

- Strong correlation between predicted and actual values
- High predictive accuracy
- Minimal bias or error
- No major outliers suggests the model generalizes well

The model is reliable and well-calibrated, making it suitable for deployment or business reporting.

Executive Summary

We tested three advanced machine learning models — XGBoost, Random Forest, and Bagging — to predict warehouse performance using historical data. After tuning each model for accuracy and reliability, XGBoost emerged as the best performer.

- It consistently gave the most accurate predictions on unseen data.
- It explained over 99% of the variation in warehouse outcomes.
- It had the lowest error rates, meaning its predictions are close to reality. The model relies heavily on one key factor: whether storage issues were reported in the last 3 months. This single feature drives most of the predictive power.

Business Recommendations

Prioritize Storage Issue Tracking, since recent storage issues are the strongest predictor of warehouse performance, ensure this data is captured accurately and consistently.

- Consider automating alerts or dashboards around this metric.
- Use the XGBoost Model for Forecasting
- Deploy the tuned XGBoost model to predict warehouse risk or performance.
- Integrate it into our reporting tools to support proactive decision-making.
- Simplify Data Collection as many other features (certifications, zones, equipment) had minimal impact.
- Focus data efforts on the few variables that truly matter — this reduces complexity and improves efficiency.
- Monitor Model Stability because the model depends heavily on one feature, keep an eye on its quality and consistency.
- Revalidate the model periodically to ensure it continues to perform well.
- Communicate Insights Clearly and share the model's findings with warehouse managers and operations teams.
- Use simple visuals (like the observed vs predicted plot) to build trust and understanding.

This approach helps us move from reactive response to predictive, data-driven warehouse management — saving time, reducing risk, and improving service.